ADVANCED MACHINE LEARNING + DEEP LEARNING PROJECT

# Microclimate Modelling via Neural Surrogates and Computational Fluid Dynamics

## Phase 1 — Machine Learning Evaluation

Project Report

Dataset: UrbanTALES — Idealized Building Arrays

Team / Project Code: T006

### Submitted by


| Team member | Student ID |
|---|---|
| Suhaani Garg | 2401010462 |
| Jashvita Lakshmi Omkaram | 2401010310 |
| Aarya Srivastava | 2401010008 |
| Palak | 2401010042 |
| Samarth Sangtani | 2401010408 |


### Repository

https://github.com/aarya-sr/T006-urban-microclimate-modelling

**Reporting note:** The supplied report draft did not include the executed model-output files. Numerical comparisons that cannot be verified from the supplied material are marked as pending rather than estimated.

# Abstract

This report documents Phase 1 (Machine Learning) of the project "Microclimate Modelling via Neural Surrogates and Computational Fluid Dynamics". Using the idealized building-array subset of the UrbanTALES computational fluid dynamics (CFD) dataset, we build a scalar surrogate that predicts the canopy-averaged wind speed (m/s) of a simulated case from layout-level geometric summaries and wind direction. The data comprise 224 simulation cases (112 unique layouts × two wind directions, 0° and 45°) grouped into 16 plan families.

The report covers every step of the pipeline: dataset selection and download, joining of the raw metadata and flow-data tables, auditing of the supplied profile, topology and NetCDF archives, data validation, exploratory analysis, feature engineering with a leakage audit, group-aware validation design, and four models (training-fold mean baseline, Ridge regression, Random Forest, XGBoost). The primary evaluation holds out entire plan families, because the project brief requires generalisation to unseen geometries. Layout-held-out and leave-one-family-out protocols, a feature-set ablation and family-clustered bootstrap uncertainty intervals support the comparison.

The project uses family-held-out validation as its primary test of generalisation. The exact model ranking and full metric comparison must be populated from a fresh execution of ML_05–ML_07. The supplied report draft does not contain the saved result tables, so no model winner is claimed here. The scalar surrogate predicts one canopy-averaged wind-speed value and cannot represent local wake patterns or street-canyon effects.

# Contents

**Abstract** i

**1 Introduction** 1

1.1 Background and Motivation . . . 1

1.2 Project Brief and Two-Phase Structure . . . 1

1.3 Problem Statement . . . 1

1.4 Research Questions . . . 2

1.5 Scope . . . 2

**2 Literature Review** 3

2.1 Background Reading . . . 3

2.2 Why These Models . . . 4

**3 Dataset** 5

3.1 Requirements and Choice of Dataset . . . 5

3.2 About UrbanTALES Idealized Arrays . . . 5

3.3 Download Procedure . . . 5

3.4 Raw Files Used . . . 6

**4 Data Extraction and Loading** 7

4.1 Repository and Path Handling . . . 7

4.2 Loading the Tabular Sources . . . 7

4.3 Auditing the ZIP Archives . . . 7

**5 Data Cleaning, Joining and Validation** 8

5.1 Building the Layout-Level Table . . . 8

5.2 Derived Identifiers . . . 8
5.3 Target Join . . . 8
5.4 Missing Values, Duplicates and Outliers . . . 9
5.5 Saved Outputs . . . 9

**6 Exploratory Data Analysis and Target Definition 10**

6.1 Target Definition . . . 10
6.2 Analyses Performed in ML_02 . . . 10
6.3 Findings . . . 11

**7 Feature Engineering and Leakage Audit 12**

7.1 Engineered Features . . . 12
7.2 Final Predictor Sets . . . 12
7.3 Leakage Controls . . . 12
7.4 Redundancy Check . . . 13

**8 Validation Design and Preprocessing 14**

8.1 Why Grouped Validation . . . 14
8.2 Protocols . . . 14
8.3 Preprocessing Pipelines . . . 14

**9 Modelling 15**

9.1 Training-Fold Mean Baseline . . . 15
9.2 Ridge Regression (ML_05) . . . 15
9.3 Random Forest and XGBoost (ML_06) . . . 15
9.4 Prediction Procedure . . . 16

**10 Evaluation Methodology 17**

10.1 Metrics . . . 17
10.2 Experiments . . . 17

**11 Results 18**

11.1 Primary Result: Family-Held-Out 5-Fold CV . . . . . 18
11.2 Effect of Validation Protocol . . . . . 18
11.3 Feature Ablation (family_5fold) . . . . . 19
11.4 Ridge Regularisation Sensitivity . . . . . 19
11.5 Random Forest versus XGBoost . . . . . 19
11.6 Feature Importance and Error Diagnostics . . . . . 19
11.7 Discussion . . . . . 19

**12 Implementation Notes, Issues and Reproducibility 20**

12.1 Environment . . . . . 20
12.2 Issues Encountered and Fixes . . . . . 20
12.3 Run Order and Outputs . . . . . 21

**13 Limitations 22**

**14 Conclusion and Phase 2 Outlook 23**

**15 Team Contributions 24**

**16 Final Submission Checklist 25**

**References 26**

**A Data Dictionary of the Modelling Table 27**

Chapter 1

# Introduction

## 1.1 Background and Motivation

Wind conditions inside urban canopies govern pedestrian comfort, ventilation and heat exposure. CFD simulation resolves these flows well but is expensive, so only a small number of layouts can be simulated. A fast surrogate that predicts flow quantities from a layout description could allow many more configurations to be screened. The project brief frames the study as one about surrogate accuracy and uncertainty, not safety-critical design approval.

## 1.2 Project Brief and Two-Phase Structure

The brief defines a hybrid logic that moves from geometric summary features to spatial surrogate fields:

- **AML baseline (Phase 1, this report):** describe each layout with building density, height statistics, street orientation and weather inputs, and predict an area-averaged wind quantity using Random Forest or XGBoost.
- **Identified gap:** layout summaries cannot represent local wake patterns, street-canyon effects or detailed spatial variation.
- **DL model (Phase 2):** encode a rasterised layout and conditions as channels and train a compact U-Net or conditional CNN to predict a velocity or temperature field.
- **Fair comparison:** hold out entire layouts rather than random pixels or rows; report field MAE, normalised RMSE and error maps for unseen geometries.

## 1.3 Problem Statement

How well can layout-level geometric summaries and wind direction predict the canopy-averaged wind-speed scalar for idealized UrbanTALES cases, especially when an entire plan family is held out?

## 1.4 Research Questions

1. How are target values and geometry features distributed, and what data-quality limitations matter?
2. Do geometry summaries and wind-direction features carry predictive signal beyond a training-fold mean baseline?
3. Do nonlinear models improve over a regularised linear model under family-held-out evaluation?
4. How much do scores change when the split is made by family versus by layout?
5. Which claims are supported by a scalar target, and which spatial-flow claims remain out of scope?

## 1.5 Scope

Phase 1 evaluates a scalar baseline only. The raster-to-field U-Net or CNN is a Phase 2 direction and is not claimed to be implemented here. The target is one wind-speed value per simulation, not a velocity map, and the dataset is an airflow dataset, so no temperature claims are made.

Chapter 2

# Literature Review

## 2.1 Background Reading

The faculty brief lists two suggested pre-reads, and a peer-reviewed review of machine learning in fluid mechanics was added for context.

- **Goodfellow et al. (2014), *Generative Adversarial Nets*.** Introduces adversarial generative modelling. It is background on learning distributions; a GAN is not needed for this scalar regression task and is not used.
- **Farimani et al. (2017), *Deep Learning for Wall Bounded Turbulent Flows*.** Listed in the brief; it motivates data-driven modelling of flow behaviour. Full bibliographic metadata should be checked against the faculty-supplied reference before final submission, so no venue or DOI is invented here.
- **Brunton, Noack and Koumoutsakos (2020), *Machine Learning for Fluid Mechanics*, Annual Review of Fluid Mechanics 52, 477–508.** Places regression, reduced-order modelling and other ML methods in the fluid-mechanics context and stresses that model choice, physical knowledge and generalisation must be considered together.

**Implication for Phase 1.** Begin with transparent scalar regression baselines and a defensible validation protocol. The brief itself names Random Forest and XGBoost on geometric summaries as the AML baseline. Spatially structured CNN or U-Net learning requires paired spatial fields, aligned grids and field-level metrics, and belongs to the later phase.

## 2.2 Why These Models

Table 2.1: Models compared and the role each plays

| Method | Role | What it tests | Limitation |
|---|---|---|---|
| **Training-fold mean** | No-feature reference | Whether features add value at all | Cannot represent layout or direction effects |
| **Ridge regression** | Linear model with L2 penalty | Whether mostly linear relationships suffice; shrinkage controls coefficient size | Limited nonlinear interaction capacity |
| **Random Forest** | Bagged trees with feature subsampling | Nonlinear thresholds and interactions | Weak extrapolation; impurity importance can be biased |
| **XGBoost** | Sequential gradient-boosted trees | Whether boosting lowers error on tabular features | Hyperparameter-sensitive; importance is not causal |

Ridge minimises the residual sum of squares plus an L2 penalty, RSS$+\alpha \sum \beta^2$, which discourages unstable large coefficients when predictors are correlated. Standardisation matters because the penalty otherwise acts unevenly on differently scaled variables.

Chapter 3

# Dataset

## 3.1 Requirements and Choice of Dataset

The project needs more than images of buildings: building geometry and CFD outputs (or enough information to generate them), inlet conditions, layout identifiers that allow layout-wise splitting, and simulation metadata. The brief recommends UrbanTALES idealized building arrays as a CFD alternative to a self-generated OpenFOAM sweep, and the project rules allow a dataset that is comparable or more complex than the one provided.

Table 3.1: Project requirements and where each is met

| Requirement | What was needed | Where it comes from |
|---|---|---|
| **Urban geometry** | Building shapes, spacing and heights | `metadata_ideal.json` (Heights, areaDensity, configuration fields) |
| **Simulation output** | Wind quantity per case | `UrbanTALES-flowdata.csv`, column `U` |
| **Input conditions** | Wind direction | `metadata_ideal.json`, Wind Direction |
| **Layout identification** | IDs to separate arrangements | Folder Name; layout and family IDs derived from it |
| **Metadata** | Density class, configuration metadata | `metadata_ideal.json` and `metadata.csv` |

## 3.2 About UrbanTALES Idealized Arrays

The idealized subset contains 224 simulated building-layout cases covering different building densities, aligned and staggered arrangements, building-height patterns and wind directions, with CFD data for urban airflow. It is an airflow dataset: it is a strong match for wind modelling but not a temperature dataset.

## 3.3 Download Procedure

The download page lists 224 simulation-case folders with filters and downloadable categories (flow data, metadata, Readme). The steps taken were:

1. Opened the UrbanTALES “Idealized Building Blocks” download page. The dataset table

initially showed "Loading…" to the AI assistant used for guidance, so the page content was copied manually to confirm the 224 case folders.

2. Applied the filters Horizontal Configuration, Standard Deviation of Building Height, Wind Direction and Plan Area Density, all set to "All", so that aligned and staggered layouts, sparse to dense arrangements and all height patterns were covered.
3. Reviewed the file types on offer: `.nc` (NetCDF multidimensional fields), `.csv` (flow statistics), `_topo` (layout grids), `_p3d` (3D data) and `.png` (visualisations only, which lose numerical field values). The initial plan was to prioritise `.nc` and `.csv` together with geometry, metadata and the Readme.
4. Obtained the metadata (`metadata_ideal.json`, `metadata.csv`), the flow-data table (`UrbanTALES-flowdata.csv`) and three sampled archives (profiles, topology, NetCDF) and placed them in `data/raw/`.

**Dataset acquisition note:** The raw files listed below are used by the pipeline. Before submission, confirm which files were downloaded directly from UrbanTALES and which were supplied by the course, and record the actual download date.

## 3.4 Raw Files Used

Table 3.2: Raw input files and their use

| File | Content | Use |
|---|---|---|
| `metadata_ideal.json` | Per-case geometry: Folder Name, Heights list, areaDensity, Wind Direction, Config, Horizontal Configuration, Plan Area Density | Defines the 224-case subset; source of all geometry features |
| `metadata.csv` | Full UrbanTALES case catalogue (NameE column) | Cross-check that all idealized IDs exist |
| `UrbanTALES-flowdata.csv` | Flow statistics per case (NameE, U, …) | Source of the target (column `U`) |
| `selected_files.zip` | Sampled vertical profile CSVs (z, U) | Diagnostic cross-check of the target |
| `selected_files (1).zip` | Sampled topology grids | Audit of layout grids; Phase 2 input candidate |
| `selected_files (2).zip` | Sampled NetCDF field files | Audit of spatial fields; Phase 2 candidate |

Chapter 4

# Data Extraction and Loading

All extraction is implemented in notebook ML_01 and is fully reproducible from the raw files; the processed CSV and data dictionary must never be edited by hand.

## 4.1 Repository and Path Handling

Every notebook locates the repository root by walking up from the working directory until it finds both `data/raw` and the Machine Learning Pipeline folder, so Jupyter can be started from either location. Outputs go to `data/processed` (datasets) and `results` (model outputs).

## 4.2 Loading the Tabular Sources

- The idealized-case JSON, the catalogue CSV and the flow-data CSV are read independently and inspected (shape, columns, sample rows) before any join, so the join assumptions are explicit.
- The script raises a clear `FileNotFoundError` if any required raw file is missing.

## 4.3 Auditing the ZIP Archives

Each archive is read in memory with `zipfile` and `BytesIO`, so no manual extraction is needed and the originals stay intact.

- **Profiles:** each CSV with columns `z` and `U` is parsed. The profile is integrated from the ground to the mean building height with the trapezoidal rule, using interpolation at the end points, and divided by the mean height. This gives a profile-integrated mean wind speed that is compared with the flow-table `U` as a diagnostic (`profile_target_crosscheck.csv`).
- **Topology:** each member is parsed as a numeric grid with `numpy.loadtxt`; rows, columns and the first 12 unique values are recorded (`topology_archive_manifest.csv`).
- **NetCDF:** files are opened with `h5py` (NetCDF4/HDF5). If that fails with `OSError`, they are read as classic NetCDF with `scipy.io.netcdf_file`. Format, dataset count and variable shapes are recorded (`netcdf_archive_manifest.csv`).

Archive coverage values and the profile-versus-flow-table MAE are generated by ML_01. Add the values from the saved archive audit outputs after a successful run.

Chapter 5

# Data Cleaning, Joining and Validation

## 5.1 Building the Layout-Level Table

Each entry in `metadata_ideal.json` becomes one row. The simulation ID is the Folder Name; the layout ID removes the wind-direction suffix (`case_id.rsplit("_d", 1)[0]`); wind direction and plan-area density are read from the metadata. The unit of observation is one layout under one wind direction.

## 5.2 Derived Identifiers

- **Family ID:** extracted with the regular expression `^[UV]([AS]\d+)` from the layout ID, i.e. the arrangement letter (A aligned, S staggered) and plan-area-density code. This gives 16 families used as the primary validation groups.
- **Height pattern:** "uniform" if the layout ID starts with U, otherwise the code after the first hyphen (CL, HR or SM). The result is stored as `configuration` = `idealized-<pattern>`.
- The catalogue label (Config) is kept as `configuration_metadata_label` for traceability but excluded from predictors, because the CL/SM labels are inconsistent in the catalogue; the folder-code configuration is used instead.

## 5.3 Target Join

The target is joined by simulation ID (NameE in the flow table) with `validate="one_to_one"`, never by row order. Checks that stop the notebook on failure:

- duplicate simulation IDs in the raw flow data;
- any case without a `U` value (missing target);
- any idealized ID absent from `metadata.csv`;
- row count other than 224, unique layouts other than 112, families other than 16, any missing cell, or wind directions other than {0, 45}.

## 5.4 Missing Values, Duplicates and Outliers

All assertions passed in the project run: the table has no missing cells, no duplicate IDs and a finite target for every case. No rows are removed. In ML_02 the 1.5×IQR rule flags potential outliers for diagnosis only, because unusual CFD outputs may be physically meaningful and must stay traceable. Imputation steps remain inside the model pipelines so the same code is valid if future data contain gaps.

## 5.5 Saved Outputs

Table 5.1: Files saved by the data-preparation notebook

| File | Description |
|---|---|
| `data/processed/layout_level_dataset.csv` | 224-row modelling table (19 columns) |
| `data/processed/data_dictionary.csv` (also `docs/`) | One row per column: units, type, meaning, source, model use; generated from the actual columns |
| `data/processed/dataset_summary.json` | Counts, target statistics, excluded features, validation notes |
| `raw_archive_manifest.csv`, `profile_target_crosscheck.csv`, `topology_archive_manifest.csv`, `netcdf_archive_manifest.csv` | Archive audit tables |

Chapter 6

# Exploratory Data Analysis and Target Definition

## 6.1 Target Definition

The target `target_U_canopy_mean_m_s` is the `U` column of the flow-data table, a canopy-averaged wind speed in m/s with one scalar per simulation. It is not a spatial velocity map.

## 6.2 Analyses Performed in ML_02

Table 6.1: Exploratory analyses and their purpose

| Step | What was done | Why |
|---|---|---|
| **Structural audit** | Row, layout and family counts; duplicate IDs; missing and infinite cells | Catch join or duplication problems before interpreting plots |
| **Numeric summaries** | `describe()` with 5/25/50/75/95 percentiles and skewness; IQR outlier screening | Spread, skew and extremes; screening only, no deletion |
| **Group summaries** | Case counts and target statistics by wind direction, configuration, horizontal configuration, density class; family-level summary | Balance of the design; shows why random row splits are risky |
| **Plot A** | Target histogram (mean and median lines) and boxplot | Shape and range of the target |
| **Plot B** | Histograms of numeric predictors | Narrow or constant predictors limit learning |
| **Plot C** | Target versus geometry scatterplots with linear guide line; Pearson correlations | Trends and nonlinearity; descriptive only |
| **Plot D** | Boxplots of target by wind direction and categorical variables | Group overlap and spread |
| **Plot E** | Paired within-layout change between 0° and 45° | Shows the two direction rows of a layout are related |
| **Plot F** | Correlation heatmap and mean target by family | Redundancy and between-family variation |

## 6.3 Findings

The exact target summary, strongest observed correlation, mean direction-related change and family-mean range must be copied from ML_02 outputs after a fresh run.

The six EDA figures (target distribution, numeric feature distributions, target–geometry relationships, categorical comparisons, within-layout direction changes and correlation/family summaries) are generated by ML_02. Insert the saved figures from a fresh notebook run here before final submission.

Chapter 7

# Feature Engineering and Leakage Audit

Feature engineering expresses plausible geometry and condition information; it cannot add CFD information or restore spatial detail discarded by aggregation.

## 7.1 Engineered Features

Table 7.1: Engineered features, definitions and rationale

| Feature | Definition | Rationale |
|---|---|---|
| `plan_area_density_sq` | plan_area_density$^2$ | Simple nonlinear density response for linear models |
| `wind_direction_sin` / `_cos` | sin and cos of direction in radians | Circular variable: 0° and 360° are neighbours; $\sin^2 + \cos^2 = 1$ is asserted |
| `height_coefficient_variation` | `height_std` / `height_mean` (0 if mean is 0) | Height dispersion relative to typical height |
| `height_std`, max, min, range, skew | Population statistics of the building-height list | Vertical heterogeneity descriptors |
| `frac_taller_than_mean` | Share of buildings taller than the mean | Distribution of tall elements |

## 7.2 Final Predictor Sets

**Primary engineered set (14 predictors):** 11 numeric (`plan_area_density`, `plan_area_density_sq`, `height_std_m`, `height_max_m`, `height_min_m`, `height_range_m`, `height_skew`, `frac_taller_than_mean`, `wind_direction_sin`, `wind_direction_cos`, `height_coefficient_variation`) and 3 categorical (`configuration`, `horizontal_configuration`, `density_class`; one-hot encoded).

**Baseline ablation set (9 predictors):** `plan_area_density`, `height_std_m`, `height_max_m`, `building_count`, `wind_direction_sin`, `wind_direction_cos` plus the same three categoricals.

## 7.3 Leakage Controls

- An assertion confirms that no forbidden field is a predictor: the target, `simulation_id`, `layout_id`, `family_id`, `configuration_metadata_label` and the raw `wind_direction_deg`.

- Identifiers and family and layout IDs are used only to build held-out groups.
- `height_mean_m` (constant) and `building_count` are retained in the table but excluded from the primary set; `building_count` appears only in the baseline ablation.
- All learned preprocessing statistics (imputation, scaling, encoding) are fitted on training folds only.

## 7.4 Redundancy Check

Add the redundant-predictor pair table generated by ML_03 after a fresh run.

The result is saved as `feature_engineered_dataset.csv` with a matching `feature_engineered_data_dictionary.csv` whose rows are asserted to match the table columns and their order.

Chapter 8

# Validation Design and Preprocessing

## 8.1 Why Grouped Validation

Rows share geometry: each layout appears twice (two directions) and layouts share a plan family. If sibling cases fall on both sides of a split, scores look better than performance on an unseen family. The brief asks for held-out layouts, so group-aware splits are used throughout.

## 8.2 Protocols

Table 8.1: Validation protocols

| Protocol | Splitter and group | Role | Folds |
|---|---|---|---|
| `family_5fold` | `GroupKFold(5)` on `family_id` | Primary generalisation estimate | 5 |
| `family_loo` | `LeaveOneGroupOut` on `family_id` | Stricter secondary check | 16 |
| `layout_5fold` | `GroupKFold(5)` on `layout_id` | Interpolation-like check; related families can appear in train and test | 5 |

Add the fold manifest and fold-size chart generated by ML_04 after a fresh run. Confirm that train and test layouts do not overlap.

## 8.3 Preprocessing Pipelines

Table 8.2: Preprocessing by model type

| Model type | Numeric features | Categorical features |
|---|---|---|
| **Ridge** | Median imputation → StandardScaler | Most-frequent imputation → `OneHotEncoder(handle_unknown="ignore")` |
| **Random Forest, XGBoost** | Median imputation only (no scaling needed for trees) | Same as Ridge |

Everything sits inside a scikit-learn `Pipeline` and `ColumnTransformer`, so `fit()` sees training rows only and held-out rows are only transformed and predicted. The fold manifest is saved as `results/family_fold_manifest.csv`.

Chapter 9

# Modelling

## 9.1 Training-Fold Mean Baseline

Predicts the mean target of the training fold for every test row. If a model cannot beat this, the features add no value.

## 9.2 Ridge Regression (ML_05)

Ridge with $\alpha = 1.0$ is the documented baseline setting, not a claimed optimum. A prespecified sensitivity analysis over $\alpha \in \{0.01, 0.1, 1, 10, 100\}$ under the same family folds shows how shrinkage changes pooled out-of-fold error. It is explicitly not nested hyperparameter optimisation, and an alpha should not be picked from a repeatedly inspected test set; formal tuning would need nested grouped cross-validation.

## 9.3 Random Forest and XGBoost (ML_06)

Table 9.1: Fixed model settings

| Setting | Random Forest | XGBoost |
|---|---|---|
| **Trees** | `n_estimators` = 300 (ML_06); 100 (ML_07) | `n_estimators` = 200 (ML_06); 100 (ML_07) |
| **Depth / leaf** | `min_samples_leaf` = 2 | `max_depth` = 2, `min_child_weight` = 3 |
| **Sampling** | `max_features` = 0.9 | `subsample` = 0.85, `colsample_bytree` = 0.9 |
| **Learning / regularisation** | — | `learning_rate` = 0.03, `reg_lambda` = 2.0 |
| **Objective / method** | Squared error | `reg:squarederror`, `tree_method` = `hist`, `n_jobs` = 4 |
| **Seed** | `random_state` = 42, `n_jobs` = −1 | `random_state` = 42 |

Hyperparameters are fixed, reproducible baseline settings; no exhaustive search was run. Note that ML_07 uses smaller ensembles (100 trees) than ML_06 to shorten the full multi-protocol run, so small numeric differences between the two notebooks are expected. State which notebook the final reported numbers come from.

Per fold, the notebooks record MAE, RMSE and $R^2$, every out-of-fold prediction with its simulation, layout and family IDs, and the tree feature importances mapped back to original feature names (one-hot columns are merged under their categorical parent). Importance is descriptive and not causal.

## 9.4 Prediction Procedure

Predictions are always out-of-fold: each case is predicted exactly once by a model that never saw its family (asserted: every simulation ID appears once per model). Pooled predictions are then used for metrics and plots.

Chapter 10

# Evaluation Methodology

All evaluation is implemented in notebook ML_07.

## 10.1 Metrics

Table 10.1: Evaluation metrics and interpretation

| Metric | Formula / meaning | Interpretation |
|---|---|---|
| **MAE** | $\text{mean}\,\lvert y-\hat{y}\rvert$ | Typical absolute error in m/s; primary ranking metric |
| **RMSE** | $\sqrt{\text{mean}\,(y-\hat{y})^2}$ | Penalises large misses |
| **Normalised RMSE** | $\text{RMSE}/(\max y-\min y)$ | Scale-relative; the target range denominator is reported |
| $R^2$ | $1-SS_{\text{res}}/SS_{\text{tot}}$ | Can be negative when worse than predicting the mean; not an accuracy percentage |

## 10.2 Experiments

- All four models under all three protocols, using the engineered feature set.
- Feature ablation: baseline feature set versus engineered set under `family_5fold`.
- Family-clustered bootstrap (1,000 resamples of families, seed 42) for 95% MAE intervals, resampling whole families to respect clustering.
- Paired comparison of Random Forest versus XGBoost: per-case difference in absolute error, bootstrapped by family (1,000 resamples, seed 0). A positive mean difference favours XGBoost, and an interval spanning zero means the difference is uncertain.
- Diagnostics: model-comparison bar chart, observed-versus-predicted scatter, absolute error by wind direction, feature importance.

The bootstrap intervals do not remove uncertainty caused by having only 16 families.

Chapter 11

# Results

All values below come from fresh runs of ML_05–ML_07 and are saved in the results folder; fill the yellow cells from the named files.

## 11.1 Primary Result: Family-Held-Out 5-Fold CV

Table 11.1: Primary comparison under family-held-out 5-fold CV

| Model | MAE (m/s) | RMSE (m/s) | Norm. RMSE | $R^2$ | MAE 95% CI |
|---|---|---|---|---|---|
| **Mean baseline** | Pending | Pending | Pending | Pending | Pending |
| **Ridge Regression** | Pending | Pending | Pending | Pending | Pending |
| **Random Forest** | Pending | Pending | Pending | Pending | Pending |
| **XGBoost** | Pending | Pending | Pending | Pending | Pending |

Source: `results/model_comparison.csv` and `results/family_bootstrap_mae_ci.csv`.

## 11.2 Effect of Validation Protocol

Table 11.2: MAE by validation protocol

| Model | family_5fold MAE | family_loo MAE | layout_5fold MAE |
|---|---|---|---|
| **Mean baseline** | Pending | Pending | Pending |
| **Ridge Regression** | Pending | Pending | Pending |
| **Random Forest** | Pending | Pending | Pending |
| **XGBoost** | Pending | Pending | Pending |

Source: `results/metrics_all_protocols.csv`.

**Interpretation guide:** if layout-held-out error is much lower than family-held-out error, holding out sibling layouts is easier than holding out whole families, so the family result is the honest generalisation estimate.

## 11.3 Feature Ablation (family_5fold)

Table 11.3: MAE with baseline versus engineered features

| Model | MAE, baseline features | MAE, engineered features | Change |
|---|---|---|---|
| **Ridge Regression** | Pending | Pending | Pending |
| **Random Forest** | Pending | Pending | Pending |
| **XGBoost** | Pending | Pending | Pending |

## 11.4 Ridge Regularisation Sensitivity

Table 11.4: Pooled out-of-fold MAE across the $\alpha$ grid

| $\alpha$ | 0.01 | 0.1 | 1 | 10 | 100 |
|---|---|---|---|---|---|
| **Pooled MAE (m/s)** | Pending | Pending | Pending | Pending | Pending |

## 11.5 Random Forest versus XGBoost

Mean paired MAE difference (RF − XGBoost) and its 95% family-bootstrap interval: awaiting verified output from `results/rf_vs_xgb_paired.csv`.

## 11.6 Feature Importance and Error Diagnostics

List the top 5–10 features per tree model from `results/feature_importance.csv` and describe errors by wind direction. Importance reflects predictive attribution within the model and is not evidence of causation.

The model-comparison, observed-versus-predicted, error-by-direction and feature-importance figures should be inserted from fresh ML_07 outputs. Do not use manually recreated or stale plots.

## 11.7 Discussion

Complete this interpretation after a fresh run of ML_05–ML_07. Compare each model with the training-fold mean baseline, state whether nonlinear models improve on Ridge, describe the family-versus-layout validation gap, and report whether engineered features help. Do not interpret feature importance as causation. Conclusions apply only to this 224-case scalar dataset and the evaluated model settings.

Chapter 12

# Implementation Notes, Issues and Reproducibility

## 12.1 Environment

Python 3.13.5 in VS Code Jupyter notebooks on macOS; libraries: pandas, numpy, matplotlib, scikit-learn, xgboost, h5py, scipy.

## 12.2 Issues Encountered and Fixes

Table 12.1: Issues encountered during implementation

| Issue | Cause | Resolution |
|---|---|---|
| `TypeError`: `Axes.boxplot()` got an unexpected keyword argument `'labels'` | Newer Matplotlib renamed the boxplot argument | Replace `labels=` with `tick_labels=` in boxplot calls only; search each match rather than blanket-replacing |
| Download page stuck on "Loading…" | Dynamic page not readable by the assistant | Page content copied manually; case folders and filters confirmed |
| Mixed NetCDF formats | Archive holds both NetCDF4/HDF5 and classic NetCDF | `h5py` first, fall back to `scipy.io.netcdf_file` |

Compatibility fixes for the current environment: use `np.trapezoid(...)` with NumPy 2.x and `tick_labels=` rather than `labels=` in newer Matplotlib boxplot calls. Search ML_02 and ML_06 for every boxplot call. Run each notebook from top to bottom and save it after successful execution so GitHub displays its outputs.

## 12.3 Run Order and Outputs

Table 12.2: Notebook run order and key outputs

| Notebook | Purpose | Key outputs |
|---|---|---|
| **ML_00** | Literature review and problem framing | — |
| **ML_01** | Raw audit, join, dictionary | `layout_level_dataset.csv`, `data_dictionary.csv`, `dataset_summary.json`, archive manifests |
| **ML_02** | EDA and target definition | Plots A–F |
| **ML_03** | Feature engineering and leakage audit | `feature_engineered_dataset.csv` and dictionary |
| **ML_04** | Grouped splits and preprocessing | `family_fold_manifest.csv` |
| **ML_05** | Ridge regression | `ridge_fold_metrics.csv`, `ridge_oof_predictions.csv`, `ridge_model_metrics.csv` |
| **ML_06** | Random Forest and XGBoost | `tree_fold_metrics.csv`, `tree_oof_predictions.csv`, `tree_model_metrics.csv`, `tree_feature_importance*.csv` |
| **ML_07** | Full comparison and uncertainty | `model_comparison.csv`, `metrics_all_protocols.csv`, `oof_predictions.csv`, `fold_metrics.csv`, `family_bootstrap_mae_ci.csv`, `rf_vs_xgb_paired.csv`, `feature_importance.csv` |

Run the notebooks in order ML_01 to ML_07. Results files are runtime outputs and are never read as inputs by later notebooks.

Chapter 13

# Limitations

- **Scalar target:** one canopy-averaged value per case; no wake patterns, street-canyon effects or spatial variation can be predicted.
- **Small sample:** 224 cases, 112 layouts and only 16 families, so fold scores vary and confidence intervals are wide.
- **Two directions only:** 0° and 45° were sampled, so conclusions about arbitrary angles are not justified.
- **Constant mean height:** all cases share a 16 m mean height, so height effects are limited to dispersion statistics.
- **Wind only:** UrbanTALES provides airflow; no temperature or heat claims are made.
- **Fixed hyperparameters:** no nested tuning, so model rankings may shift with tuning.
- **Archive coverage:** the profile, topology and NetCDF archives are samples (32 members each) and do not cover all cases.
- **Simulation limits:** the study concerns surrogate accuracy and uncertainty on idealized cases, not safety-critical design approval.

Chapter 14

# Conclusion and Phase 2 Outlook

A reproducible Phase 1 pipeline is defined from raw UrbanTALES files to a validated 224-case layout-level table. It includes archive auditing, exploratory analysis, feature engineering, leakage controls, grouped validation and comparisons between a mean baseline, Ridge, Random Forest and XGBoost. The final conclusion about which model performs best must be based on fresh, saved results from the completed notebooks. Phase 2 can extend this scalar baseline to rasterised-layout learning with a U-Net or conditional CNN, provided sampled spatial fields are sufficient and correctly aligned.

The scalar baseline sets the reference that Phase 2 must beat. Phase 2 will use the sampled topology grids and NetCDF fields as the starting point for a rasterised-layout U-Net or conditional CNN predicting velocity fields, with layout-held-out evaluation using field MAE, normalised RMSE and error maps, and a hybrid ML–DL comparison where technically justified. This is a plan, not an implemented result.

Chapter 15

# Team Contributions

The team divided the notebook workflow so that each member has a clear area of ownership. All members should be prepared to explain the complete pipeline and the decisions made in their assigned notebooks during the individual viva.

Table 15.1: Notebook ownership by team member

| Team member | Notebook contribution |
|---|---|
| Suhaani Garg — 2401010462 | Notebook 1 — ML_01: raw-data audit, data extraction, processed dataset and data dictionary |
| Aarya Srivastava — 2401010008 | Notebook 2 — ML_02: exploratory data analysis and target understanding |
| Jashvita Lakshmi Omkaram — 2401010310 | Notebooks 3–4 — ML_03 feature engineering/leakage audit; ML_04 grouped validation and preprocessing |
| Palak — 2401010042 | Notebook 5 — ML_05: Ridge regression baseline and regularisation analysis |
| Samarth Sangtani — 2401010408 | Notebooks 6–7 — ML_06 tree models; ML_07 evaluation and model comparison |

Chapter 16

# Final Submission Checklist

- Run ML_01–ML_07 in order from the repository root using the project Python environment.
- Save each notebook after a successful run so outputs and plots are visible on GitHub.
- Confirm processed datasets and dictionaries are generated from raw inputs and are not hand-edited.
- Populate result tables and figures only from the matching fresh notebook outputs.
- Verify the repository URL, course section, faculty mentor, dataset citation, licence and actual download date.
- Ensure every team member can explain the data, features, grouped split, metrics and limitations individually.

# References


[1] I. Goodfellow et al., "Generative Adversarial Nets," *Advances in Neural Information Processing Systems*, vol. 27, 2014.

[2] A. Farimani et al., "Deep Learning for Wall Bounded Turbulent Flows," 2017. (Title and year as given in the project brief; verify full publication details against the faculty source.)

[3] S. L. Brunton, B. R. Noack and P. Koumoutsakos, "Machine Learning for Fluid Mechanics," *Annual Review of Fluid Mechanics*, vol. 52, pp. 477–508, 2020, doi: 10.1146/annurev-fluid-010719-060214.

[4] UrbanTALES dataset (idealized building arrays), documentation and datasheet supplied with the project. Add the exact dataset citation, DOI and licence from the official datasheet before submission.

## Appendix A

# Data Dictionary of the Modelling Table

Table A.1: Fields of the layout-level modelling table

| Field | Units | Meaning | Model use |
|---|---|---|---|
| simulation_id | — | Case identifier (layout + direction) | Traceability only |
| layout_id | — | Base geometry shared by its direction runs | Grouping only |
| family_id | — | Arrangement letter + density code (16 families) | Primary validation group |
| target_U_canopy_mean_m_s | m/s | Canopy-averaged wind speed (flow-data U) | Target |
| plan_area_density | ratio | Plan-area density | Numeric predictor |
| plan_area_density_sq | ratio$^2$ | Squared density | Numeric predictor |
| height_mean_m | m | Mean building height (constant, 16 m) | Not a predictor |
| height_std_m | m | Std. dev. of building heights | Numeric predictor |
| height_max_m, height_min_m, height_range_m | m | Max, min and range of heights | Numeric predictors |
| height_skew | — | Skewness of heights (0 if uniform) | Numeric predictor |
| frac_taller_than_mean | fraction | Share of buildings taller than mean | Numeric predictor |
| wind_direction_deg | degrees | Inflow direction (0 or 45) | Retained; replaced by sin/cos |
| wind_direction_sin, wind_direction_cos | — | Circular encoding of direction | Numeric predictors |
| height_coefficient_variation | — | height_std / height_mean | Numeric predictor |
| configuration | category | Height pattern (uniform, CL, HR, SM) | One-hot predictor |
| configuration_metadata_label | category | Original catalogue label | Excluded (inconsistent) |
| horizontal_configuration | category | Aligned or staggered | One-hot predictor |
| density_class | category | Density regime from metadata | One-hot predictor |
| building_count | count | Buildings in the height list | Baseline ablation only |